

Problem Set – Week 17: MLOps

Overview

This week's assignment applies MLOps discipline – environment reproducibility, experiment tracking, and monitoring – to the work you've already built. There are **two tracks, and you're required to complete both**: Track A applies MLOps to a classical ML pipeline, Track B applies it to the assistant you built in W15/W16.

- **Track A – Data Science MLOps**: apply MLOps to a trained predictive model, using the shared dataset specified below.
- **Track B – Agentic AI MLOps**: apply MLOps to your W15 AI Assistant and W16 Agentic Feature.

Both tracks require the same three tools, applied to different kinds of systems:

Tool	Data Science (Track A)	Agentic AI (Track B)
uv	Reproducible Python env for the ML pipeline	Reproducible Python env for the assistant
MLflow	Track training runs, register best model	Track agent/prompt configuration runs
Evidently AI	Data & model drift on tabular data	Regression Test/Evaluation
Airflow (<i>optional</i>)	Scheduled retrain/drift-check DAG	Scheduled regression-eval DAG

You will be required to submit a link to your GitHub repository. You may either use a separate repository for each track or maintain a single repository with different feature branches for each track.

Your repository should be **well-organized and thoroughly documented**, including clear instructions on how to set up, run, and understand your work.

Please note that your submission will also be evaluated and marked based on these criteria, including the quality of documentation, organization of the repository, implementation, and overall completeness of the project.

Track A: Data Science MLOps

Objective

Bring a predictive ML pipeline up to production MLOps standards: reproducible environment, tracked experiments, a justified model choice, a registered model, and drift monitoring.

Dataset

Use the **Telco Customer Churn** dataset (IBM sample dataset, widely mirrored on Kaggle: [blastchar/telco-customer-churn](https://www.kaggle.com/blastchar/telco-customer-churn)) – ~7,000 customers, a mix of categorical and numeric features, target: binary **Churn** (Yes/No).

Core Functionality

1. Environment Management (uv)

- Migrate your project to **uv** – a [pyproject.toml](https://docs.pyproject.org/en/latest/build-system/uv.html) and committed [uv.lock](https://docs.pyproject.org/en/latest/build-system/uv.html).
- One-command reproducible setup ([uv sync](https://docs.pyproject.org/en/latest/build-system/uv.html)).

2. Experiment Tracking (MLflow)

- Instrument your training script to log to MLflow: parameters, metrics, and artifacts.
- Train **at least 3 models with different hyperparameters** on the churn dataset (e.g., vary [max_depth/n_estimators](https://mlflow.org/docs/latest/python_api/mlflow.pyfunc.html) for a tree-based model, [C/penalty](https://mlflow.org/docs/latest/python_api/mlflow.pyfunc.html) for logistic regression, or compare model families entirely – your choice, as long as the hyperparameters genuinely differ between runs, not just the random seed).
- For every run, log: the hyperparameters used, classification metrics (at minimum accuracy, precision, recall, F1, and ROC-AUC – churn datasets are imbalanced, so accuracy alone is not sufficient), and artifacts (the model file, confusion matrix, and ROC curve).
- Use the MLflow UI (or an exported run-comparison table/screenshot) to compare all runs side by side.
- Register the best-performing model in the **MLflow Model Registry**, and transition it through at least two stages (e.g., [Staging](https://mlflow.org/docs/latest/python_api/mlflow.pyfunc.html) → [Production](https://mlflow.org/docs/latest/python_api/mlflow.pyfunc.html)).

3. Model Serving

- Serve the registered model behind a simple API (MLflow's built-in [mlflow models serve](https://mlflow.org/docs/latest/python_api/mlflow.pyfunc.html), or a thin FastAPI/Flask wrapper that loads the model from the registry).

4. Monitoring (Evidently AI)

- Split the Telco Churn dataset into a **reference set** (e.g., a random 70% sample, treated as "training-time" data) and a **current set** (the remaining 30%, treated as "incoming production" data).
- Into the current set, **deliberately inject synthetic drift** so the drift report has something concrete to detect, for example:
 - Add random noise to a numeric column (e.g., shift **MonthlyCharges** or **tenure** by a random offset per row).
 - Skew a categorical column's distribution (e.g., oversample **Contract == "Month-to-month"** relative to its natural frequency).
 - Optionally, flip a small percentage of the target label to simulate label/concept drift as well as feature drift.
- Run Evidently's **Data Drift report** on reference vs. current and confirm it flags the specific column(s) you perturbed.
- Run Evidently's **Target Drift report** (or equivalent) to check whether the churn rate itself shifted between the two sets.
- Define **at least one custom metric** beyond Evidently's built-in defaults — e.g., the difference in mean **MonthlyCharges** between reference and current, or the shift in churn rate within a specific **Contract** segment — using Evidently's custom metric/test API.
- Briefly interpret the report: which features drifted, whether the drift you engineered was correctly detected, and what that would mean for the model if this happened in production.
- Save the generated report (generate html report) in MLflow.

5. Orchestration — Optional Bonus (Airflow)

- Build a DAG that runs the drift check on a schedule (e.g., daily/weekly/monthly) and, on detecting significant drift, triggers (or logs a recommendation to trigger) retraining.

Deliverables

- Use of **uv**
- **MLflow tracking** artifacts: full run comparison table/screenshots for **all ≥3 models**, plus registry stage transitions
- **Evidently AI** reports (HTML)
- Documentation (Readme file)
- Standard workflow:
data → training → tracking → registry → serving → monitoring → retraining (if needed)

Track B: Agentic AI MLOps

Objective

Extend the assistant and agentic feature from W15/W16 with the same three MLOps disciplines. Since there's no "trained model" here, tracking and monitoring are applied to **configurations and behavior** instead: prompts, retrieval settings, agent parameters, and the evaluation metrics from your W16 harness.

Core Functionality

1. Environment Management (uv)

- Migrate your W15/W16 assistant codebase to [uv](#) — [pyproject.toml](#) + committed [uv.lock](#)..

2. Experiment Tracking (MLflow)

- Version your system prompts explicitly (e.g., [prompt_v1](#), [prompt_v2](#), [prompt_v3](#), stored as separate files or tagged strings) — this is the primary thing you're experimenting on, alongside other configuration values (chunk size / top-k, model choice, temperature, agentic loop max-iterations).
- For each version, run your test queries and capture the full **trace** for every query, not just the final answer — a structured, step-by-step record of what the agent actually did:
 - Each tool call, with its arguments and the raw result returned.
 - The model's intermediate decision at each step (why it chose that tool, or why it decided to stop/continue).
 - The total number of iterations used before the loop terminated, and why it terminated (success, max-iterations hit, etc.).
 - You can build this yourself by having your agentic loop write each step out as a structured record (e.g., one JSON object per step: `{step, tool, args, result, reasoning}`), or use MLflow's built-in **tracing** ([mlflow.trace](#) / LLM autologging) so traces show up directly in the MLflow UI next to that run's params and metrics.
 - Save at least 2–3 representative traces per version as MLflow artifacts — ideally one clean success and one failure case, since the failure case is what actually informs your next prompt revision below.
- Use those traces to diagnose *why* a version underperforms (e.g., it stopped searching too early, misread a tool result, picked the wrong tool) and let that diagnosis drive your next prompt revision — the point is that each new version should be a response to a specific failure you saw in the traces, not a speculative tweak.
- Log whatever parameters, metrics, and artifacts you think are meaningful for comparing versions — parameters might include the prompt version and other config values you varied; metrics could pull from your W16 evaluation harness or be ones you define

yourself; artifacts might be the prompt text, sample traces, or harness reports. Use your judgment here — this is your chance to show what you think is worth tracking, not a fixed checklist.

- Run **at least 3 prompt/configuration versions**, each iterating on what the traces revealed about the previous version's failures, and use MLflow to compare which version performs best and at what cost.

3. Monitoring & Regression Testing (Evidently AI)

Evidently has native LLM evaluation support (`pip install evidently[llm]`), including LLM-as-a-judge and regression testing built.

- Build a small regression test set: a fixed list of representative queries, each paired with an approved "golden" reference response (these can come from your best-performing W16/earlier prompt version).
- Whenever you change the prompt (or model, retrieval config, etc.), re-run the same queries through the new version to get new responses.
- Use Evidently's Test Suite with an LLM-as-a-judge evaluator to compare new responses against the reference set. Define at least two checks for the judge:
 - Reference-based correctness: does the new response contradict or lose information present in the reference answer? (Evidently's `BinaryClassificationPromptTemplate` is built for exactly this — the judge returns "correct"/"incorrect" plus its reasoning.)
- Run the Test Suite after every prompt change and treat a failing test as a real regression, not just a metric that moved — if too many test cases fail, that prompt version shouldn't be promoted.
- Log the Test Suite's pass/fail results as an MLflow metric for that run (e.g., `pct_tests_passed`), so your MLflow comparison across prompt versions includes this regression signal alongside your harness numbers.
- Interpret the results: which specific test cases failed and why, and whether the judge's verdicts line up with what you see when you read the responses yourself — a sanity check on the judge, since an LLM judge can itself be wrong or biased.

4. Orchestration — Optional Bonus (Airflow)

- Build a DAG that runs your W16 evaluation harness on a schedule (e.g., nightly regression eval) and flags (or alerts on) metric degradation beyond a defined threshold.

Deliverables

- Updated source code (W15 assistant + W16 agentic feature + this week's changes)

- `pyproject.toml` + `uv.lock`
 - MLflow tracking artifacts (exported run comparison across configurations)
 - Evidently AI reports (HTML)
 - Documentation
-

Documentation Requirements (README) – Both Tracks

As with W16, these sections are part of the assessment and must reflect **your** implementation, not a general explanation of the tools.

a. Environment & Reproducibility (uv) State what dependency/version problems `uv` solves for your project specifically, and confirm the one-command reproduction path works from a clean clone.

b. Experiment Tracking Strategy (MLflow) Explain what you varied across runs and why, what you measured, and what the comparison revealed – which configuration/model won, and what the trade-off was (e.g., accuracy vs. latency, or task completion vs. token cost).

Track A specifically: this section must justify **which of your ≥ 3 models you registered and why**, and that justification must be backed by the actual MLflow run comparison – include the metrics table (or screenshot) for all runs side by side, not just the winning run, and reference the specific numbers that drove the decision (e.g., "Run 2 had the highest F1 (0.81) and ROC-AUC (0.85) despite Run 1's marginally higher accuracy, because accuracy is misleading on this imbalanced target"). A stated preference without the comparison to back it up will not satisfy this section.

c. Monitoring & Drift Strategy (Evidently AI) Explain what "reference" vs. "current" means in your setup, what metric(s) you monitored, what the report showed, and what action you (or an automated system) would take if drift crossed a concerning threshold.

d. Orchestration (*only if you implemented the Airflow bonus*) Describe the DAG: schedule, trigger condition, and what happens on a positive drift/degradation signal.

Submission Checklist

1. Github link for both tracks (Track A ML pipeline + Track B assistant), You may create different repo for each track or keep it in single repo using different branches.

2. `pyproject.toml` + `uv.lock` for each track
3. MLflow run comparisons for both tracks, + registered model info (Track A)
4. Evidently AI HTML report(s) for both tracks
5. Airflow DAG file(s) (if attempted, either or both tracks)